



Tiffin Tracker

Complete Project Guide: how it works, every screen, every term, explained simply

Updated with morning notifications, subscription transfer and messy customer import.

What this document covers

1. The idea in one page
2. The big picture: how the pieces fit together
3. The billing rules (the heart of the project)
4. End-to-end workflow: a tiffin owner's month
5. Website tour: every page and component, with screenshots
6. New features: morning notifications, subscription transfer, messy import
7. Backend: how a click becomes saved data
8. The database, table by table
9. The REST API, endpoint by endpoint
10. Security, validation and errors
11. Glossary: every term used in the website and code
12. Running, testing and troubleshooting

Repository: github.com/Nikunj532/tiffin-tracker · Stack: React + Node.js/Express + SQLite

1 The idea in one page

A **tiffin service** is a home-style lunch delivery business. Customers pay a **monthly plan price** and get lunch delivered **every weekday** (Monday to Friday).

Real life gets in the way. A customer travels for a week, goes home for Diwali, or falls sick. They **pause** the tiffin and shouldn't pay for those days. When they're back, they **resume**.

At month-end the owner needs to answer "*How much does each customer owe?*" Doing that by hand with a notebook means counting weekdays and subtracting paused days for every customer, and mistakes are common.

What Tiffin Tracker does

Owner's need	How the app solves it
Keep a list of customers	Customers page: add, edit, search, sort, page through
Know who is on which plan	Plans page plus subscriptions linking a customer to a plan
Record pauses	Pause / Resume buttons on each customer's page
Bill fairly	Automatic pro-rated bill: pay only for days actually served
Find a customer quickly when they call	Phone lookup on the Dashboard
See who is active vs paused	Status badges, status filter tabs, Dashboard counts
Work for any tiffin business	Each owner registers their own account; data is kept separate

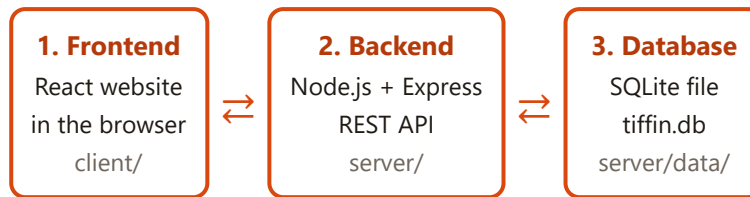
Who uses it

Only the **tiffin owner** logs in. Customers never see the app; the owner records their pauses when they call or message. The landing page lists "customer self-service pauses" as a future feature.

One-line summary: subscribe → pause/resume → month-end bill = plan price × days delivered ÷ weekdays in month.

2 The big picture

The project is a **full-stack web application**, which means it has three layers that talk to each other:



Layer	What it is	Its job, in plain words
Frontend	React app (JavaScript that runs in the browser)	Draws the screens: buttons, tables, forms. When you click something it <i>asks</i> the backend for data or to save data. It never touches the database directly.
Backend	Node.js program using the Express library	The "brain and gatekeeper". It checks you're logged in, checks the data makes sense, applies the billing rules, and reads or writes the database.
Database	SQLite, a database stored in one file	The permanent memory. Customers, plans, pauses and bills survive restarts because they're saved here.

How they communicate

The frontend and backend talk using **HTTP requests** carrying **JSON** (a simple text format for data). Example: when you open the Customers page, the browser sends:

```
GET /api/customers?status=paused&page=1&limit=10&sort=name&order=asc
Authorization: Bearer eyJhbGciOi... ← your login token
```

and the backend replies with:

```
{ "data": [ { "id": 1, "name": "Aarav Sharma", "phone": "9845000000", "status": "paused", ... } ],
  "page": 1, "limit": 10, "total": 8, "totalPages": 1,
  "counts": { "active": 17, "paused": 8, "upcoming": 3, "inactive": 4 } }
```

Folder structure

```
tiffin-tracker/
├── client/                                ← FRONTEND (React)
│   └── src/
│       ├── main.jsx                      routes: which URL shows which page
│       ├── api.js                        helper that calls the backend + formats ₹ and dates
│       ├── auth.js                       remembers who is logged in
│       ├── userListQuery.js              keeps search/sort/page in the URL
│       ├── components/                  Layout (top bar) and shared UI pieces
│       └── pages/                        Landing, Auth, Dashboard, Customers, CustomerDetail, Plans, Bills,
                                           Notifications, ImportCustomers
├── server/                               ← BACKEND (Express)
│   ├── src/
│   │   ├── index.js                     starts the server on port 4000
│   │   ├── app.js                       connects all routes + error handling
│   │   ├── db.js                        creates the database tables
│   │   ├── billing.js                   THE BILLING FORMULA (pure function)
│   │   ├── dates.js                     date helpers (weekday? add days? month bounds?)
│   │   ├── http.js                      login check, pagination, error helpers
│   │   ├── seed.js                      fills the database with demo data
│   │   ├── clock.js                     simulated clock + morning job runner
│   │   ├── notifications.js             Notification Service (outbox) + "due today" rule
│   │   ├── importer.js                  CSV parsing, date/phone cleaning, row checks
│   │   └── routes/                       auth, plans, customers, subscriptions, bills, dashboard,
                                           simulation (clock + outbox), import
│   ├── test/                            automated tests
│   └── data/tiffin.db                    ← DATABASE FILE (created automatically)
├── samples/messy-customers.csv           example messy list for the importer
├── README.md REASONING.md AI_LOGS.md
└── package.json                          shortcut commands (install:all, seed, build, start, test)
```

In **production mode** (`npm start`) the backend also serves the built website, so everything runs at one address: **http://localhost:4000**. In **development mode** the website runs separately on port 5173 and forwards `/api` calls to port 4000.

3 The billing rules

Everything in this project exists to make this calculation correct. It lives in one file, `server/src/billing.js`, in a function called `computeBill`.

$$\text{Bill} = \text{Plan price} \times \text{Delivered days} \div \text{Weekdays in the month}$$

Step by step

1. **Count the weekdays in the month** (Monday–Friday). September 2026 has **22**. This is the "Weekdays in month" number.
2. **Find the subscribed weekdays**: weekdays that fall between the subscription's start date and end date (if any). If a customer started on the 1st or earlier and hasn't stopped, that's all 22. If they started on 22 Sep, it's only 7.
3. **Remove paused weekdays**: any subscribed weekday inside a pause. Saturdays and Sundays are ignored because nothing is delivered then anyway.
4. **Delivered days** = subscribed weekdays – paused weekdays.
5. **Bill** = price × delivered ÷ weekdays in month, rounded to the nearest paisa.

Worked example: Aarav Sharma, September 2026

Plan: **Veg Mini, ₹2,500/month**. Paused **15 Sep to 20 Sep** (Travel).

M	T	W	T	F	S	S	Weekdays in September	22
	1	2	3	4	5	6	Subscribed weekdays	22 (subscribed since July)
7	8	9	10	11	12	13	Paused weekdays	4 (15, 16, 17, 18; the 19th and 20th are a weekend)
14	15	16	17	18	19	20	Delivered days	22 – 4 = 18
21	22	23	24	25	26	27	Bill	₹2,500 × 18 ÷ 22 = ₹2,045.45
28	29	30						

What "Weekdays 22/22" means on the Bills page

The Bills table shows **subscribed weekdays / weekdays in month**.

- **22/22**: the customer was subscribed on every one of the 22 weekdays in the month.
- **7/22**: the subscription only covered 7 of the month's weekdays (for example it started on 22 Sep), so the bill is smaller.
- **21/22**: the subscription ended or started one weekday short of the full month.

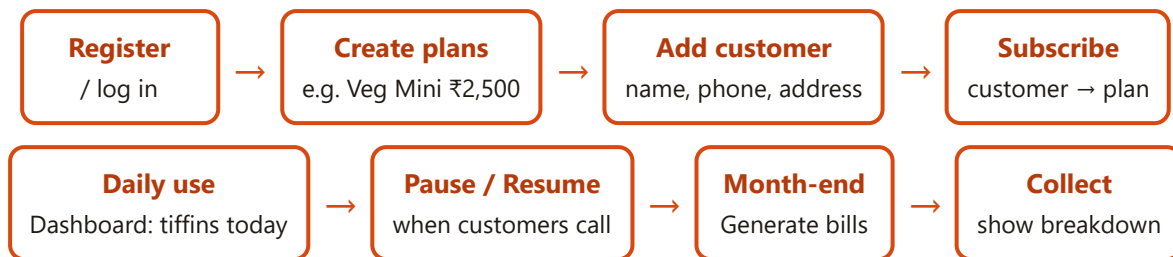
Paused days are shown separately in the **Paused** column, and **Delivered** = **subscribed** – **paused**.

The special rules, and why

Rule	Why
Divide by the month's actual weekday count (20–23)	A full month always costs exactly the plan price. Never more, never less.
Weekend days in a pause don't reduce the bill	No lunch was going to be delivered on a weekend.

Pause end date is inclusive (last paused day)	Matches how people talk: "paused 7th to 11th".
A pause with no end = "until resumed"	The customer doesn't know when they'll be back.
Resume on date D = D is the first delivery day again	The pause is closed the day before D. If D is the pause's own first day, the pause is deleted.
Pauses can't overlap	Otherwise a day could be counted as paused twice.
The plan price is copied onto the subscription	If the owner raises the plan price later, existing customers' bills don't change silently.
Money stored in paise (whole numbers)	Decimals in computers can drift ($0.1 + 0.2 \neq 0.3$). ₹2,045.45 is stored as 204545.
A pause spanning two months is split	Each month only counts the paused days that fall inside it.

4 End-to-end workflow: a tiffin owner's month



Step 1: Register and log in

The owner creates an account (name, tiffin service name, email, password). The backend stores a **hashed** password and returns a **token**. The browser keeps that token and sends it with every later request, which is how the backend knows who is asking.

Step 2: Create meal plans

On **Plans**, the owner adds plans such as "Veg Mini ₹2,500", "Veg Full Thali ₹3,500" and "Non-Veg Thali ₹4,500". A plan can be **retired** (unticked "Available for new subscriptions") so no new customers join it, while existing customers stay on it.

Step 3: Add customers

On **Customers** → + **Add customer**, enter name, phone and address. Phone numbers are cleaned (spaces and dashes removed) and must be unique for that owner, because the phone number is how customers are looked up.

Step 4: Subscribe

On the customer's page click **Subscribe to a plan**, choose the plan and a start date. The subscription stores the start date and a copy of the price. A customer can only have one running subscription at a time.

Step 5: Day to day

- The **Dashboard** shows how many tiffins to cook and deliver today (the number of Active customers on a weekday) and who is paused.
- A customer calls: type their phone into **Find customer by phone** to jump to their page.
- They're travelling: click **Pause**, pick first and last paused day (or "until I resume"), and add a reason.
- They're back early: click **Resume** and choose the first delivery day.
- They're leaving: click **End subscription** with the last delivery day.

Step 6: Month-end billing

Open **Bills**, pick the month and click **Generate bills**. The backend runs the billing formula for every subscription that touched that month and **saves** the results. The table shows each customer's weekdays, paused days, delivered days and amount, plus the total to collect.

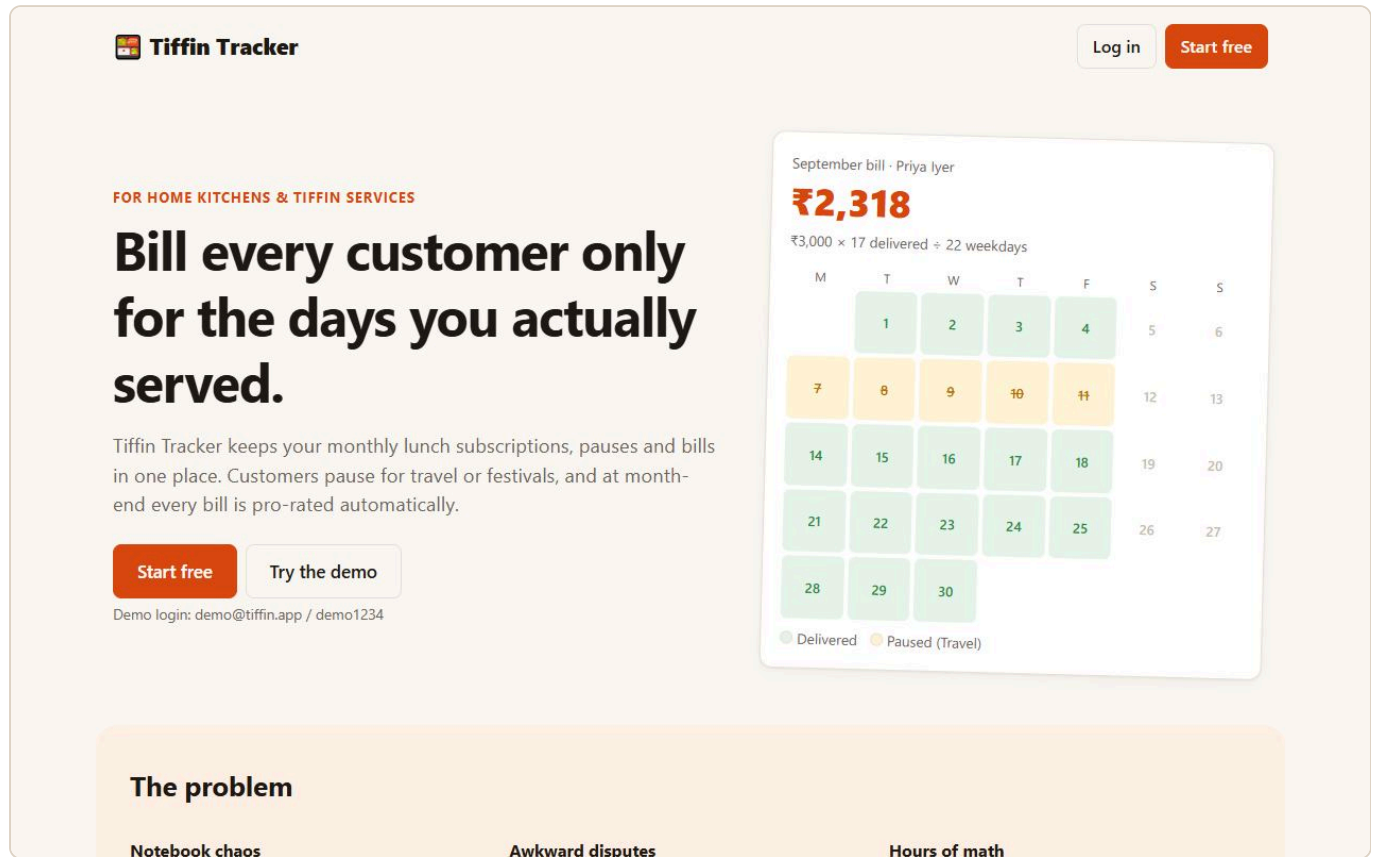
If a pause is corrected afterwards, click **Regenerate**. The saved bills are recomputed from scratch, and running it twice gives the same result.

Status updates itself. The app never stores "paused" as a flag. Every time, it checks today's date against the pauses. A pause scheduled for next week turns into "Paused" on the right morning without anyone clicking anything.

5 Website tour: every page and component

5.1 Landing page (/)

The public front page, visible without logging in. File: `client/src/pages/Landing.jsx`.



Hero section: headline, short description, "Start free" / "Try the demo" buttons, and a sample bill card with a delivery calendar.

Component	What it shows / does
Navigation bar	Logo, plus Log in and Start free buttons (shows Open dashboard if you're already logged in).
Hero	What the product is, in one sentence, with the main call-to-action buttons.
Sample bill card	A mini example of a pro-rated bill ($₹3,000 \times 17 \div 22 = ₹2,318$) with paused days struck through.
The problem	Three pain points: notebook chaos, awkward disputes, hours of math.
Key features	Six feature cards (pause/resume, fair bills, phone lookup, status, calendar, search/sort/paginate).
Who it's for / How it helps	Target audience and benefits.
Coming next	Three planned features: WhatsApp bills with UPI links, customer self-service pauses, kitchen and route sheet.
Call-to-action band and footer	A final sign-up button.

Key features



Pause & resume in one tap

Travel, Diwali at home, a sick day: log a pause with dates or leave it open. Paused days are never charged.



Fair pro-rated bills

Plan price $\times$ delivered weekdays $\div$ weekdays in the month, worked out exactly. No spreadsheets and no arguments.



Find anyone by phone

Type the number a customer calls from and their plan, pauses and bill come up right away.



Active vs paused at a glance

See how many tiffins to cook today and who is paused, upcoming or inactive.



Delivery calendar

Each customer's month shown day by day: delivered, paused, weekend, not subscribed.



Search, sort, paginate

Handles 30 customers or 3,000. Filter, sort and page through customers, plans and bills.

Who it's for

- ✓ Home cooks running a tiffin service for 10–500 customers
- ✓ Mess and dabba services delivering to offices and PGs
- ✓ Cloud kitchens with monthly meal subscriptions

How it helps

- ✓ **Accurate:** bills match what was delivered, down to the paisa
- ✓ **Fast:** month-end billing drops from hours to one click
- ✓ **Trusted:** show customers a day-by-day calendar with their bill

"Key features" cards.

- ✓ Home cooks running a tiffin service for 10–500 customers
- ✓ Mess and dabba services delivering to offices and PGs
- ✓ Cloud kitchens with monthly meal subscriptions

- ✓ **Accurate:** bills match what was delivered, down to the paisa
- ✓ **Fast:** month-end billing drops from hours to one click
- ✓ **Trusted:** show customers a day-by-day calendar with their bill
- ✓ **Organised:** know exactly how many tiffins to cook today

Coming next

WhatsApp bills + UPI links

Send each customer their monthly bill on WhatsApp with a UPI payment link, and mark bills paid automatically.

Customer self-service pauses

A private link where customers pause or resume themselves before a daily cut-off, so the owner stops fielding calls.

Kitchen & route sheet

A daily cooking count per plan and delivery routes grouped by area, printable for the delivery staff.

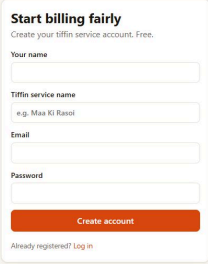
Stop losing money on paused days, and stop overcharging for them.

[Start free](#)

"Who it's for", "How it helps" and "Coming next" sections.

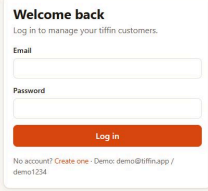
5.2 Register and Log in (/register , /login)

File: `client/src/pages/Auth.jsx` . Both are simple forms inside a card.



The register form is titled "Start billing fairly" and includes the subtext "Create your tiffin service account. Free." It contains four input fields: "Your name", "Tiffin service name" (with a hint "e.g. Max Ki Raviol"), "Email", and "Password". Below the fields is an orange "Create account" button. At the bottom, there is a link "Already registered? Log in".

Register: your name, tiffin service name, email, password (min 8 characters).



The login form is titled "Welcome back" and includes the subtext "Log in to manage your tiffin customers." It contains two input fields: "Email" and "Password". Below the fields is an orange "Log in" button. At the bottom, there is a link "No account? Create one: Demo: demo@tiffinapp / demo1234".

Log in: email and password. Shows demo credentials.

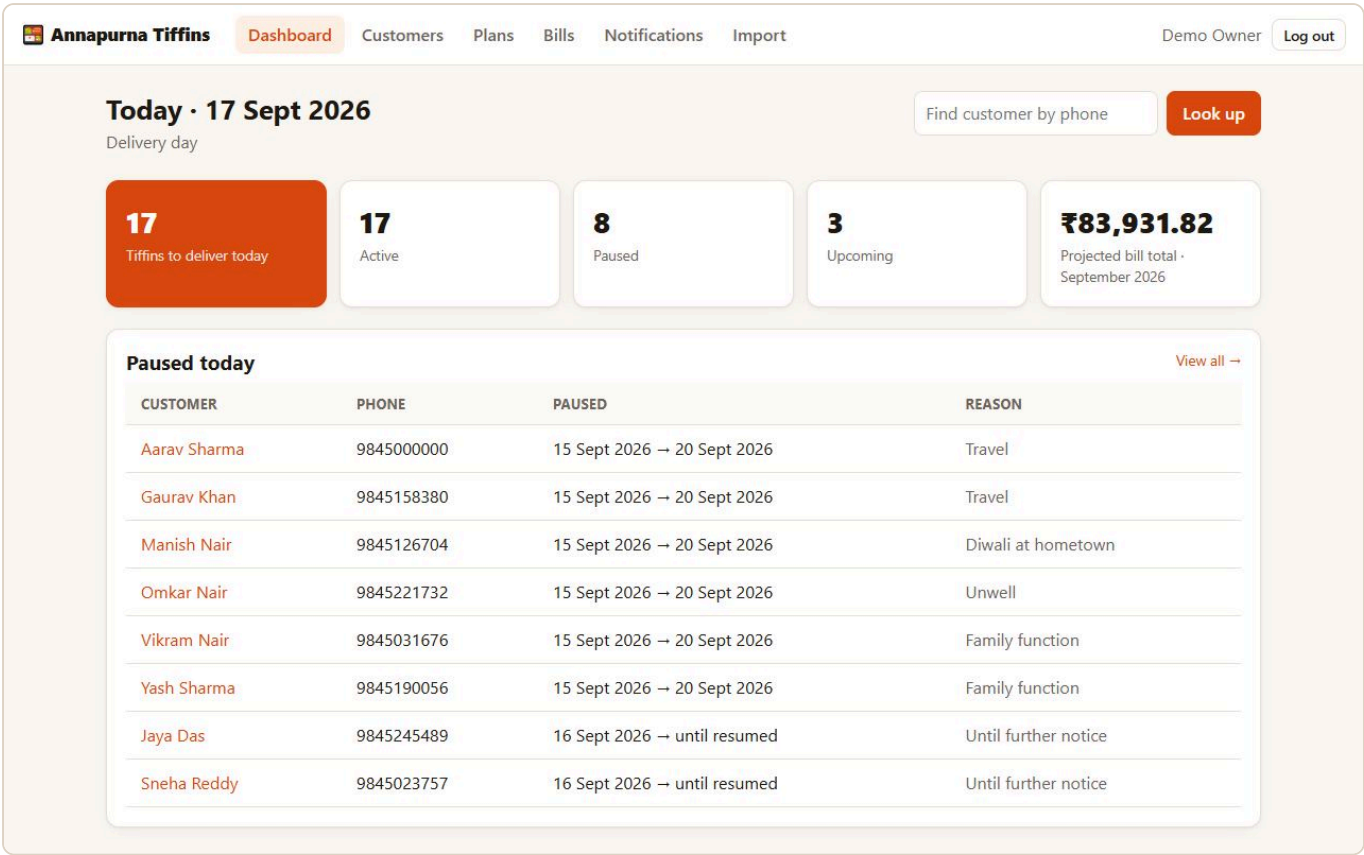
- Errors from the backend (e.g. "An account with this email already exists") appear in a red **alert box** above the form.
- After registering you're taken to **Plans** (a new owner's first job). After logging in you're taken to the **Dashboard**.
- If you're already logged in, these pages redirect you into the app (**GuestOnly** guard). App pages redirect logged-out visitors to Log in (**Protected** guard).

5.3 The app layout: top bar

File: `client/src/components/Layout.jsx` . Every page inside `/app` shares it:

- **Brand:** the owner's tiffin service name (e.g. "Annapurna Tiffins"). Click it to go to the Dashboard.
- **Navigation tabs:** Dashboard · Customers · Plans · Bills · Notifications · Import. The current page is highlighted in orange.
- **Clock badge:** appears only when the app is running on a **simulated date** (see Chapter 6). Click it to open Notifications.
- **User area:** owner's name and **Log out** (forgets the token and returns to the landing page).

5.4 Dashboard (/app)



Dashboard: today's summary, status counts, projected revenue, phone lookup and "Paused today" list.

Component	Meaning
Today · date	Today's date, and whether it's a Delivery day (weekday) or weekend.
Find customer by phone	Type a phone number and press Look up to open that customer. Shows "No customer with phone ..." if not found.
Tiffins to deliver today	Number of Active customers today (0 on weekends). This is how many lunches to cook.
Active / Paused / Upcoming cards	How many customers are in each status. Click a card to open Customers filtered to that status.
Projected bill total	What this month's bills would total if generated right now, calculated live from current pauses.
Paused today table	Customers paused today, with pause dates and reason (up to 10; "View all →" for the rest).
Get started callout	Only shown to a new owner with no plans; links to Plans.

5.5 Customers list (/app/customers)

File: `client/src/pages/Customers.jsx` . This page shows **search**, **filtering**, **sorting** and **pagination**.


Annapurna Tiffins

[Dashboard](#)
[Customers](#)
[Plans](#)
[Bills](#)
[Notifications](#)
[Import](#)

Demo Owner
[Log out](#)

Customers

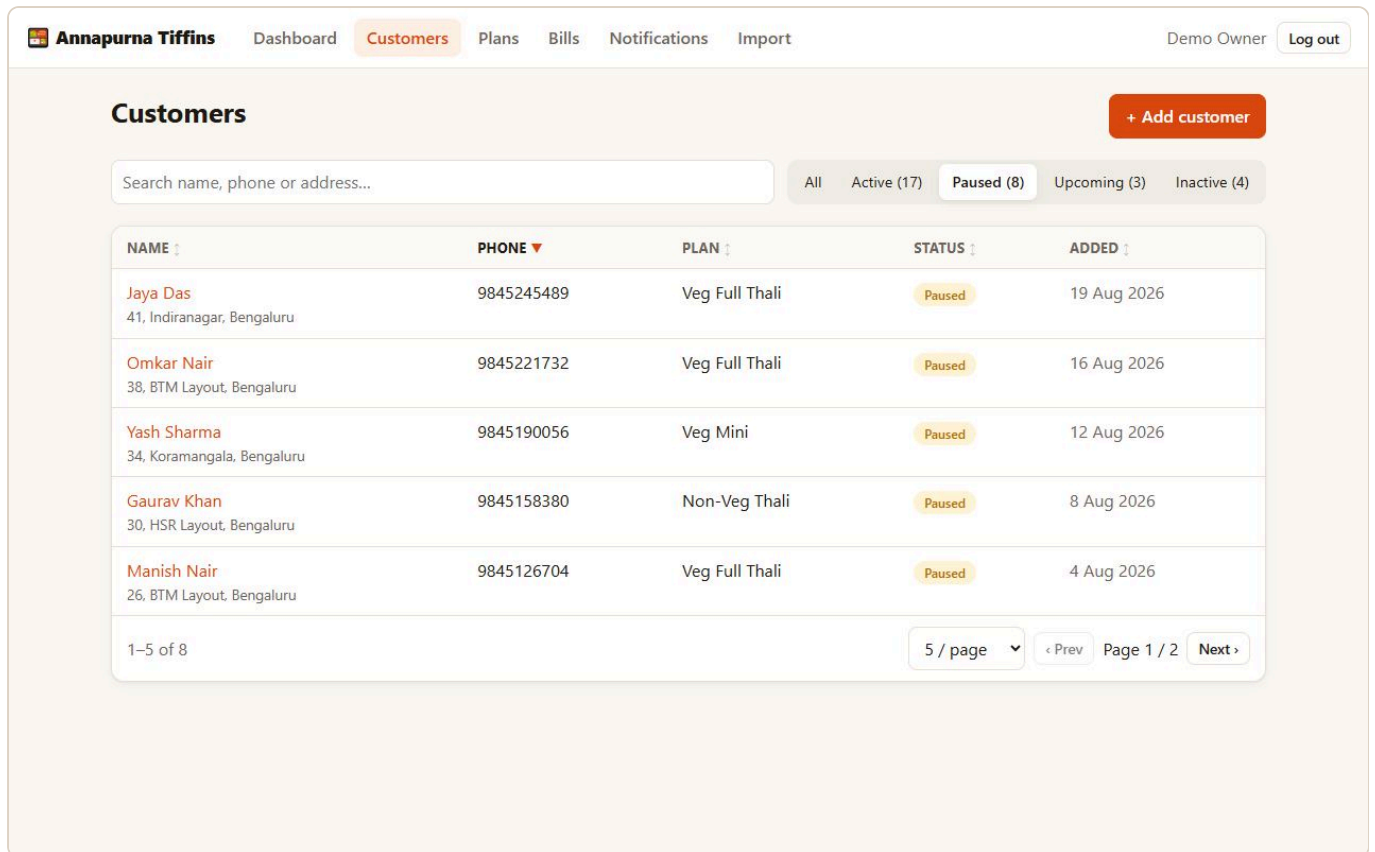
+ Add customer

All
Active (17)
Paused (8)
Upcoming (3)
Inactive (4)

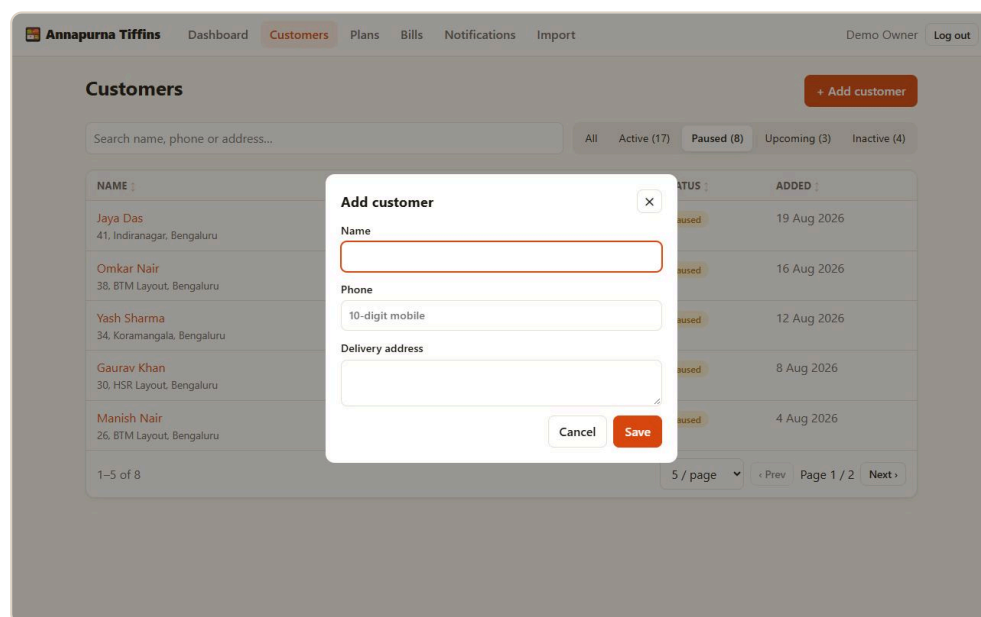
NAME ▲	PHONE ↕	PLAN ↕	STATUS ↕	ADDED ↕
Aarav Sharma 10, Koramangala, Bengaluru	9845000000	Veg Mini	Paused	19 Jul 2026
Aditya Patel 24, HSR Layout, Bengaluru	9845110866	Non-Veg Thali	Active	2 Aug 2026
Ananya Gupta 15, Jayanagar, Bengaluru	9845039595	Non-Veg Thali	Active	24 Jul 2026
Arjun Khan 18, HSR Layout, Bengaluru	9845063352	Non-Veg Thali	Upcoming	27 Jul 2026
Bhavna Reddy 37, Whitefield, Bengaluru	9845213813	Veg Mini	Active	15 Aug 2026
Deepak Kulkarni 40, Koramangala, Bengaluru	9845237570	Veg Mini	Active	18 Aug 2026
Divya Menon 19, Whitefield, Bengaluru	9845071271	Veg Mini	Active	28 Jul 2026
Farhan Kulkarni 28, Koramangala, Bengaluru	9845142542	Veg Mini	Active	6 Aug 2026
Gaurav Khan	9845158380	Non-Veg Thali	Paused	8 Aug 2026

All customers, sorted by name, 10 per page.

Component	What it does
Search box	Searches name, phone and address as you type. It waits 0.3 s after you stop typing before asking the server (debounce), so it doesn't send a request per keystroke.
Status tabs	All · Active · Paused · Upcoming · Inactive, with counts in brackets.
Sortable column headers	Click Name , Phone , Plan , Status or Added to sort. Click again to reverse. ▲ = ascending (A→Z, old→new), ▼ = descending, ↕ = not sorted by this column.
Status badge	Coloured pill: green active , yellow paused , blue upcoming , grey inactive .
Pagination bar	"1–10 of 32", rows per page (5/10/20/50), < Prev, Page X / Y, Next >.
+ Add customer	Opens the Add customer modal (pop-up form).



Filtered to **Paused**, sorted by **Phone** ▼, **5 per page**. The URL becomes `?status=paused&sort=phone&order=desc&limit=5`, so refreshing or sharing the link keeps the same view.



Add customer modal: name, phone (10–13 digits), delivery address. After saving you're taken to the new customer's page.

5.6 Customer detail (/app/customers/:id)

File: `client/src/pages/CustomerDetail.jsx`. Everything about one customer, and where pauses happen.

Annapurna Tiffins

Dashboard
Customers
Plans
Bills
Notifications
Import

Demo Owner
Log out

Customers

Aarav Sharma
Paused

9845000000
10, Koramangala, Bengaluru

Edit
Delete

Veg Mini · ₹2,500/month

Since 4 Jul 2026 · Paused until 20 Sept 2026

Resume
Schedule pause
Transfer
End subscription

Pauses

FROM	TO	REASON	
15 Sept 2026	20 Sept 2026	Travel	Remove

Bill for September 2026

September, 2026

₹2,045.45

₹2,500 × 18 delivered ÷ 22 weekdays in month

Weekdays in month 22
Subscribed weekdays 22
Paused weekdays 4
Delivered 18

M	T	W	T	F	S	S
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30				

Delivered
Paused
Weekend
Not subscribed

Aarav Sharma: status, subscription card with actions, pause history, and this month's bill with a delivery calendar.

Component	What it does
Header	Name, status badge, phone, address; Edit (same form as Add) and Delete (asks for confirmation; removes all their history).
Subscription card	Plan name and price, "Since" start date, and "Paused until ..." if currently paused.
II Pause	Shown when not paused. Opens the pause form.
► Resume	Shown when paused today. Opens the resume form.
Schedule pause	Shown while paused, to add another future pause.
End subscription	Sets the last delivery day. Pauses after it are trimmed.
Pauses table	Every pause (From, To or "until resumed", Reason). Remove deletes a pause entered by mistake, so those days become billable again.
Subscribe to a plan	Shown instead of the card when the customer has no running subscription.
Bill for <month>	A live preview of the bill. Pick any month with the month picker. Shows the amount, the formula with real numbers, and the four counts.
Delivery calendar	Each day coloured: green = delivered, yellow struck-through = paused, faded = weekend, grey = not subscribed (before start or after end).
Subscription history	Shown only if the customer had more than one subscription over time.

Pause deliveries

First paused day: 17-09-2026

Last paused day: dd-mm-yyyy

☐ Pause until I resume manually

Reason (optional):

Travel, festival, unwell...

Paused weekdays are not billed.

Cancel Pause

Pause form: first paused day, last paused day, "Pause until I resume manually" checkbox, reason.

Resume deliveries

First delivery day: 17-09-2026

The pause ends the day before this date.

Cancel Resume

Resume form: first delivery day. "The pause ends the day before this date."

Subscribe Rahul Joshi

Plan: Non-Veg Thali - ₹4,500/month

Start date: 17-09-2026

Cancel Subscribe

Subscribe form for a customer without a plan: choose an active plan and start date.

What happens on a mistake? If you try to pause dates that overlap an existing pause, the form shows e.g. "Overlaps an existing pause starting 2026-09-15 to 2026-09-16" and nothing is saved.

5.7 Plans (/app/plans)

 **Annapurna Tiffins**

[Dashboard](#) [Customers](#) [Plans](#) [Bills](#) [Notifications](#) [Import](#)

Demo Owner [Log out](#)

Meal plans

Monthly price covers every weekday. Price changes apply to new subscriptions only.

+ New plan

Search plans...

PLAN ▲	MONTHLY PRICE ▴	SUBSCRIBERS ▴	STATUS	
Non-Veg Thali Thali with chicken/egg curry	₹4,500	10	Available	Edit Delete
Veg Full Thali 4 rotis, rice, dal, 2 sabzi, salad	₹3,500	9	Available	Edit Delete
Veg Mini 3 rotis, dal, sabzi	₹2,500	9	Available	Edit Delete

1–3 of 3

10 / page ▾ < Prev Page 1 / 1 Next >

Meal plans with price, number of current subscribers and availability.

- **+ New plan / Edit:** name, what's included, monthly price in ₹ (converted to paise when saved), and "Available for new subscriptions".
- **Status "available" / "retired":** retired plans can't be chosen for new subscriptions.
- **Delete:** only allowed if nobody ever subscribed to the plan; otherwise retire it.
- Search box, sortable columns (Plan, Monthly price, Subscribers) and pagination work like the Customers page.

5.8 Bills (/app/bills)

 Annapurna Tiffins

[Dashboard](#) [Customers](#) [Plans](#) [Bills](#) [Notifications](#) [Import](#)

Demo Owner

Log out

Monthly bills

September, 2026

Regenerate

Plan price × delivered weekdays ÷ weekdays in month. Paused days are never charged.

₹83,931.82

Total to collect · September 2026

30

Bills

531

Tiffins delivered

46

Paused days (not charged)

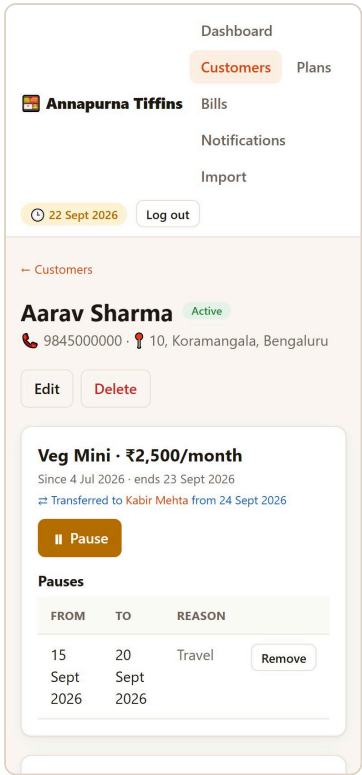
Last generated 17/9/2026, 4:34:25 pm

CUSTOMER ▲	PLAN ↓	WEEKDAYS	PAUSED ↓	DELIVERED ↓	AMOUNT ↓
Aarav Sharma 9845000000	Veg Mini ₹2,500/mo	22/22	4	18	₹2,045.45
Aditya Patel 9845110866	Non-Veg Thali ₹4,500/mo	22/22	0	22	₹4,500
Ananya Gupta 9845039595	Non-Veg Thali ₹4,500/mo	22/22	0	22	₹4,500
Arjun Khan 9845063352	Non-Veg Thali ₹4,500/mo	7/22	0	7	₹1,431.82
Bhavna Reddy 9845213813	Veg Mini ₹2,500/mo	22/22	0	22	₹2,500
Deepak Kulkarni 9845237570	Veg Mini ₹2,500/mo	21/22	0	21	₹2,386.36

Monthly bills: month picker, Regenerate, summary cards, searchable and sortable bill table.

Component	Meaning
Month picker	Choose which month's bills to view or generate.
Generate bills / Regenerate	Calculates and saves bills for every subscription active in that month.
Total to collect	Sum of all bill amounts for the month.
Bills	Number of bills (one per subscription that touched the month).
Tiffins delivered	Sum of delivered days across all customers, i.e. the total lunches served.
Paused days (not charged)	Sum of paused weekdays: lunches customers didn't pay for.
Last generated	When the bills were last calculated. If pauses changed since then, click Regenerate.
Customer column	Name (links to the customer page) and phone.
Plan column	Plan name and the monthly price used for the bill.
Weekdays column	subscribed weekdays / weekdays in month , e.g. 22/22 (whole month) or 7/22 (joined late).
Paused column	Paused weekdays in that month.
Delivered column	Subscribed – paused. The number the customer pays for.
Amount column	The final bill in ₹.

5.9 Works on phones too



Customer page on a phone-sized screen.

The layout is **responsive**: on narrow screens, columns stack vertically, less important table columns (Plan, Added) are hidden, and tables scroll sideways instead of breaking the page.

6 New features: notifications, transfer, import

Three features were added on top of the core product. Each one reuses the same rules as billing, so everything stays consistent.

Level	Feature	In one sentence
1	Morning notifications	Each morning, every customer who will get a tiffin today is sent a message through the Notification Service.
2	Subscription transfer	Hand a running subscription to another person mid-month; plan and cycle continue, and each person pays for their own days.
3	Messy customer import	Upload a messy spreadsheet; the app cleans it into subscriptions and reports what was imported, deduped and rejected.

6.1 Level 1: morning delivery notifications

The problem

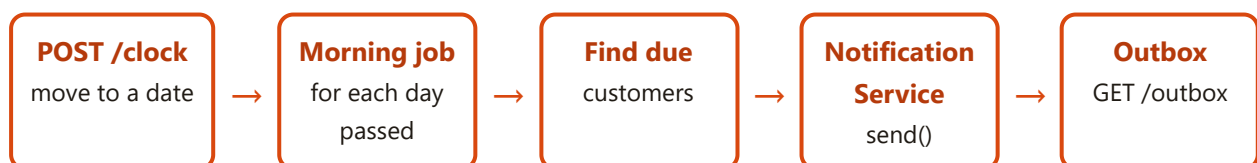
Customers like a reminder that lunch is coming, but they must **not** be told "delivery today" on a weekend, while paused, or before their plan starts.

Who is "due a delivery today"?

Due today = subscription running today AND today is Mon–Fri AND no pause covers today

This is exactly the rule billing uses for "delivered days", so nobody is ever reminded about a lunch they won't be charged for.

How it works



- **Simulated clock:** real mornings can't be waited for in a demo, so `POST /clock` sets the app's "today". The **whole app** follows it: statuses, dashboard, default dates and the 🕒 badge.
- **Catch-up:** jumping from Friday to Tuesday runs Saturday, Sunday, Monday and Tuesday in order, so Monday's reminders aren't skipped. Weekends send nothing.
- **No duplicates:** each message has a unique key (*subscription* + *date*). Running the same morning again sends 0 new messages.
- **Notification Service:** one function, `send()`, that every message goes through. Today it writes to the **outbox** table; a real SMS/WhatsApp provider could be plugged in behind it later.
- **Survives restarts:** the simulated date is saved in the database.

The Notifications page (/app/notifications)

Dashboard

Customers

Plans

Bills

Notifications

Import

🕒 21 Sept 2026

Demo Owner

Log out

Notifications

Each morning, customers due a delivery (active, a weekday, not paused) get a reminder through the Notification Service.

🕒

Clock · 21 Sept 2026

Simulated date (real date 17 Sept 2026). The whole app uses this date.

21-09-2026

Jump to date

Back to real date

Jumping forward runs every morning in between, so nobody's reminder is skipped. Running a morning twice never sends duplicates.

Next morning ▶

Run this morning

Last run: 40 new notifications

MORNING	DUE	SENT	ALREADY SENT
18 Sept 2026	17	17	0
19 Sept 2026 Weekend	0	0	0
20 Sept 2026 Weekend	0	0	0
21 Sept 2026	23	23	0

Search phone, name or message...

dd-mm-yyyy

All types

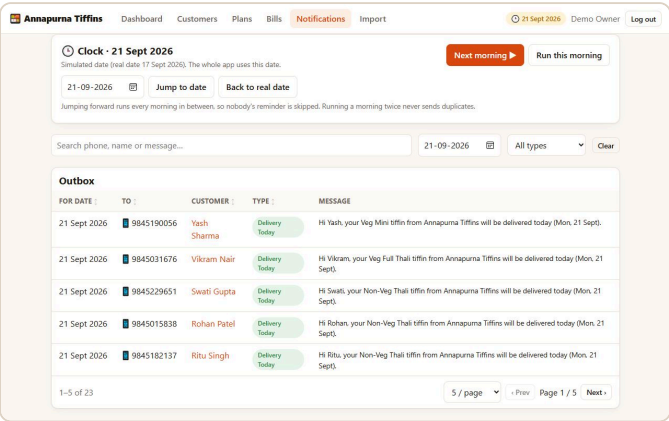
Outbox

FOR DATE	TO	CUSTOMER	TYPE	MESSAGE
21 Sept 2026	📞 9845190056	Yash Sharma	Delivery Today	Hi Yash, your Veg Mini tiffin from Annapurna Tiffins will be delivered today (Mon, 21 Sept).
21 Sept 2026	📞 9845031676	Vikram Nair	Delivery Today	Hi Vikram, your Veg Full Thali tiffin from Annapurna Tiffins will be delivered today (Mon, 21 Sept).
21 Sept 2026	📞 9845229651	Swati Gupta	Delivery Today	Hi Swati, your Non-Veg Thali tiffin from Annapurna Tiffins will be delivered today (Mon, 21 Sept).
21 Sept 2026	📞 9845015838	Rohan Patel	Delivery Today	Hi Rohan, your Non-Veg Thali tiffin from Annapurna Tiffins will be delivered today (Mon, 21 Sept).
21 Sept 2026	📞 9845182137	Ritu Singh	Delivery Today	Hi Ritu, your Non-Veg Thali tiffin from Annapurna Tiffins will be delivered today (Mon, 21 Sept).
21 Sept 2026	📞 9845007919	Priya Iyer	Delivery Today	Hi Priya, your Veg Full Thali tiffin from Annapurna Tiffins will be delivered today (Mon, 21 Sept).
21 Sept 2026	📞 9845102947	Pooja Iyer	Delivery Today	Hi Pooja, your Veg Full Thali tiffin from Annapurna Tiffins will be delivered today (Mon, 21 Sept).

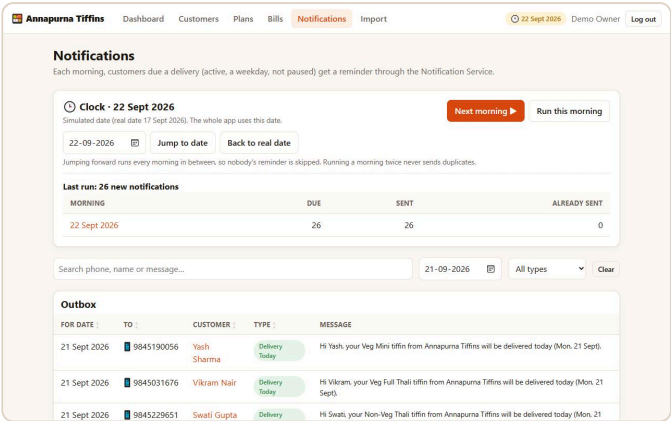
Jumped from 17 Sep to Mon 21 Sep: the app ran 18th (17 sent), 19th and 20th (weekend, 0), 21st (23 sent). The outbox lists every message.

Component	What it does
Clock card	Shows the current app date and whether it is simulated (with the real date for reference).
Next morning ▶	Moves the clock forward one day and runs that morning.
Run this morning	Runs today's job again. Only customers not yet notified today get a message (useful after adding a customer).
Jump to date	Moves to any date. Going forward runs every morning in between; going back runs only that date.
Back to real date	Stops simulating and uses the computer's real date.

Last run table	Per morning: Due (customers due), Sent (new messages), Already sent (skipped as duplicates), and a weekend badge. Click a date to filter the outbox.
Outbox filters	Search phone/name/message, filter by date or type, clear filters.
Outbox table	For date, To (phone), Customer, Type badge, Message. Sortable, paginated, newest first.



Outbox filtered to 21 Sep, 5 per page.



After "Next morning": Tue 22 Sep, 23 due, 23 sent.

For graders/testing: POST /clock and GET /outbox also work at the root URL without logging in. Because the outbox contains phone numbers, set PUBLIC_SIM_ENDPOINTS=false in a real deployment to require login.

6.2 Level 2: transfer a subscription mid-cycle

The problem

A customer moves out of their PG on the 24th and a flatmate takes over their tiffin. The flatmate should continue the **same plan at the same price** without starting a new cycle, and each person should pay only for the lunches they ate.

How it works



1. Pick the **new holder's first delivery day** (the effective date) and the person taking over: an **existing customer** (found by phone) or a **new customer** (name, phone, address). If the phone already exists, that customer is reused.
2. The old subscription **ends the day before**. Pauses after that day are removed (they were the old holder's plans).
3. A **new, linked subscription** is created with the **same plan**, the **same locked price** (even if the plan price changed since) and the **same end date** if one was set.
4. Both customers get a message in the outbox (*Transferred out* / *Transferred in*).
5. The screen shows the **billing split** for that month.

How the bill splits

No new formula is needed. Each subscription is billed for its own served weekdays, over the same "weekdays in month":

September 2026 (₹2,500 plan, 22 weekdays)	Served weekdays	Paused	Delivered	Bill
Aarav (1–23 Sep)	17	4	13	$₹2,500 \times 13 \div 22 = \text{₹1,477.27}$
Kabir (24–30 Sep)	5	0	5	$₹2,500 \times 5 \div 22 = \text{₹568.18}$
Combined	22	4	18	₹2,045.45 , the same as Aarav alone would have paid

Transfer subscription

Aarav Sharma's **Veg Mini** (₹2,500/month) passes to someone else mid-cycle. The plan, price and cycle carry over; each person is billed only for the weekdays they were served.

New holder's first delivery day

24-09-2026

Aarav Sharma's last delivery will be the day before. Their pauses after that are dropped.

Existing Customer **New Customer**

Name

Kabir Mehta

Phone

98765 11111

Delivery address

22, Koramangala, Bengaluru

Cancel **Transfer**

Transfer dialog: first delivery day for the new holder, then an existing customer (by phone) or a new customer.

Annapurna Tiffins

DashboardCustomersPlansBillsNotificationsImport

22 Sept 2026Demo OwnerLog out

Customers

Aarav SharmaActive

9845000000 · 10, Koramangala, Bengaluru

EditDelete

Transferred to Kabir Mehta from 24 Sept 2026 (new customer created)

SEPTEMBER 2026	SERVED WEEKDAYS	PAUSED	DELIVERED	BILL
Aarav Sharma	17/22	4	13	₹1,477.27
Kabir Mehta	5/22	0	5	₹568.18
Combined				₹2,045.45

Veg Mini · ₹2,500/month

Since 4 Jul 2026 · ends 23 Sept 2026

Transferred to Kabir Mehta from 24 Sept 2026

Pause

Pauses

FROM	TO	REASON	
15 Sept 2026	20 Sept 2026	Travel	Remove

Bill for September 2026

September, 2026

₹1,477.27

₹2,500 × 13 delivered ÷ 22 weekdays in month

Weekdays in month22

Subscribed weekdays17

Paused weekdays4

Delivered13

M T W T F S S

1 2 3 4 5 6

7 8 9 10 11 12 13

14 15 16 17 18 19 20

21 22 23 24 25 26 27

28 29 30

Delivered Paused Weekend Not subscribed

After transferring: green summary with the split, "⇌ Transferred to Kabir Mehta" note, and the calendar greys out days after 23 Sep for Aarav.

Annapurna Tiffins

DashboardCustomersPlansBillsNotificationsImport

22 Sept 2026Demo OwnerLog out

Customers

Kabir MehtaUpcoming

9876511111 · 22, Koramangala, Bengaluru

EditDelete

Veg Mini · ₹2,500/month

Since 24 Sept 2026

Taken over from Aarav Sharma on 24 Sept 2026

Pause⇌ TransferEnd subscription

Pauses

No pauses. Every weekday is billed.

Bill for September 2026

September, 2026

₹568.18

₹2,500 × 5 delivered ÷ 22 weekdays in month

Weekdays in month22

Subscribed weekdays5

Paused weekdays0

Delivered5

M T W T F S S

1 2 3 4 5 6

7 8 9 10 11 12 13

14 15 16 17 18 19 20

21 22 23 24 25 26 27

28 29 30

Delivered Paused Weekend Not subscribed

Annapurna Tiffins

DashboardCustomersPlansBillsNotificationsImport

22 Sept 2026Demo OwnerLog out

Bills

Monthly bills

September, 2026Regenerate

Plan price × delivered weekdays ÷ weekdays in month. Paused days are never charged.

₹83,931.82

Total to collect · September 2026

31

Bills

531

Tiffins delivered

46

Paused days (not charged)

Search by customer name or phone...

Last generated 17/9/2026, 4:34:55 pm

CUSTOMER	PLAN	WEEKDAYS	PAUSED	DELIVERED	AMOUNT
Aarav Sharma 9845000000	Veg Mini ₹2,500/mo ⇌ to Kabir Mehta	17/22	4	13	₹1,477.27
Aditya Patel 9845110866	Non-Veg Thali ₹4,500/mo	22/22	0	22	₹4,500
Ananya Gupta 9845039595	Non-Veg Thali ₹4,500/mo	22/22	0	22	₹4,500
Arjun Khan 9845063352	Non-Veg Thali ₹4,500/mo	7/22	0	7	₹1,431.82
Bhava Reddy 9845213813	Veg Mini ₹2,500/mo	22/22	0	22	₹2,500

1 - 5 of 315 / pagePrevPage 1 / 7Next

Kabir's page: "⇌ Taken over from Aarav Sharma", same plan and price. Bills page: Aarav's row shows "⇌ to Kabir Mehta" (Kabir's shows "⇌ from Aarav").

Rules that protect the data

Situation	Result
Effective date on or before the subscription start	400: must be mid-cycle
Subscription already ended before the effective date	400: nothing left to transfer
Transfer to the same customer	400
New holder already has a running subscription	409 Conflict
Anything fails half-way	Nothing is saved (one transaction)

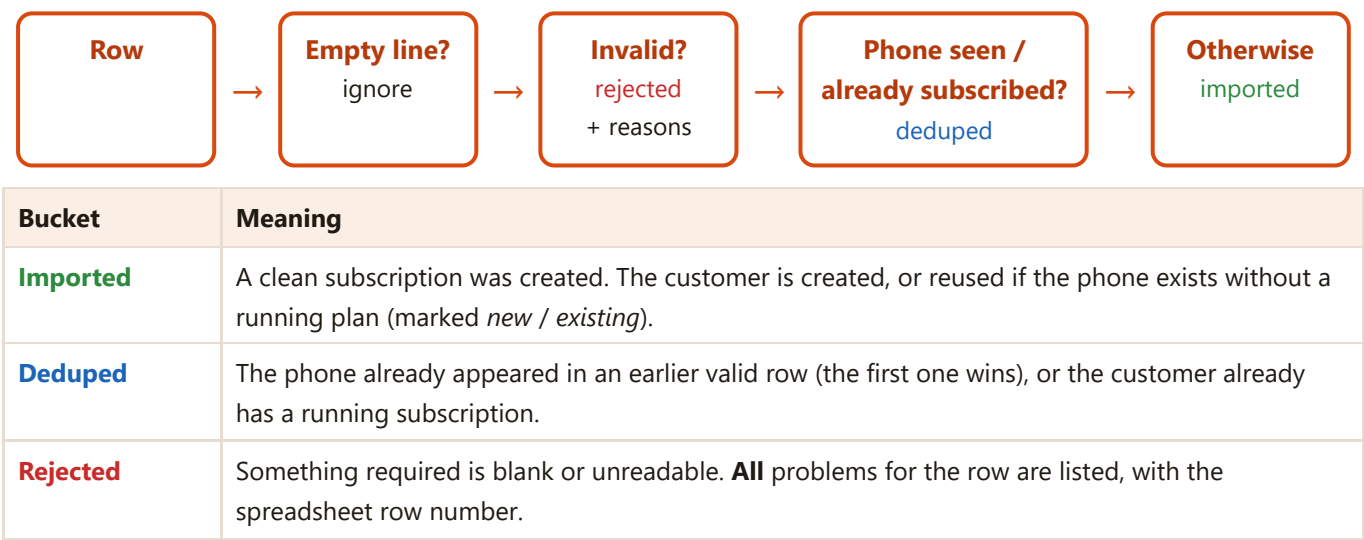
The ⇌ **Transfer** button disappears once a subscription has been transferred.

6.3 Level 3: import a messy customer list

The problem

Owners keep customers in spreadsheets typed by different people: phones as "+91 98450 00000" or "098450-00000", dates as "01/09/2026", "1 Sep 2026" or "2026-09-01", names in CAPITALS, the same customer twice, and blank cells.

Every row ends up in exactly one bucket



What gets cleaned automatically

Field	Messy input	Becomes
Column headers	Customer Name, Mobile No, Meal Plan, Start Date	name, phone, plan, start_date (unknown columns like "Notes" are ignored and listed)
Phone	+91 92000 00001 · 92000-00002 · 092000000006	9200000001 · 9200000002 · 9200000006
Name	ASHA IYER · asha iyer	Asha Iyer (mixed case like "McDonald" is left alone)

Plan	veg mini · VEG FULL THALI	matched to your plans ignoring case and extra spaces
Date (year first)	2026-09-01 · 2026/9/1 · 20260901	2026-09-01
Date (day first)	01/09/2026 · 1-9-26 · 01.09.2026	2026-09-01 (ambiguous ones get a warning)
Date (month first)	09/25/2026	2026-09-25, only because 25 can't be a month
Date (words)	1 Sep 2026 · 1st September 2026 · Sep 25 2026	ISO date
Impossible date	31/02/2026	rejected (never silently turned into 3 March)

Optional defaults: choose a plan and/or start date to use when those cells are blank (each use is noted as a warning). **Preview** shows the full report without saving. The real import is one transaction, and importing the same file twice is safe: everything becomes "deduped".

The Import page (/app/import)

Annapurna Tiffins

Dashboard
Customers
Plans
Bills
Notifications
Import

22 Sept 2026
Demo Owner
Log out

Import customers

Paste or upload a customer list, even a messy one. Phones are cleaned, dates in mixed formats are read, duplicates are merged, and bad rows are listed with reasons.

Choose CSV file

or paste below

Load messy sample

```

Customer Name,Mobile No,Address,Meal Plan,Start Date,Notes
ASHA IYER,+91 92000 00001,"12, MG Road, Indiranagar",veg mini,01/09/2026,caps name + +91 phone + dd/mm date
Ravi Kumar,92000-00002,,Non-Veg Thali,2026-09-03,dashes in phone
asha iyer,92000000001,,Veg Mini,5 Sep 2026,duplicate phone of row 2
,9200000003,HSR Layout,Veg Mini,2026-09-01,blank name
Meena,,,Veg Mini,2026-09-01,blank phone
Kiran,92000000004,,Veg Mini,,blank start date
Tara,9200000005,,Pizza Plan,2026-09-01,unknown plan
Om,12345,,Veg Mini,31/02/2026,bad phone and impossible date

```

Columns are matched by name (name/customer, phone/mobile, address, plan, start_date/start). Dates like 2026-09-01, 01/09/2026, 1 Sep 2026 and Sep 1, 2026 all work: ambiguous dd/mm are read day-first. Plans are matched to **your plans** by name.

Plan when blank (optional)

Start date when blank (optional)

Reject rows without a plan

dd-mm-yyyy

Preview (no changes)
Import

Choose a CSV file or paste text (or "Load messy sample"), set optional defaults, then Preview or Import.

Annapurna Tiffins

DashboardCustomersPlansBillsNotificationsImport

22 Sept 2026Demo OwnerLog out

Columns are matched by name (name/customer, phone/mobile, address, plan, start_date/start). Dates like 2026-09-01, 01/09/2026, 1 Sep 2026 and Sep 1, 2026 all work; ambiguous dd/mm are read day-first. Plans are matched to your plans by name.

Plan when blank (optional)

Reject rows without a plan

Start date when blank (optional)

dd-mm-yyyy

Preview (no changes)Import

Preview only: nothing has been saved yet. 11 rows read, 1 empty line(s) ignored, ignored columns: Notes.

Import 5 now

5

Imported

1

Deduped

5

Rejected

ROW	CUSTOMER	PHONE	PLAN	START	NOTES
2	Asha Iyer New	9200000001	Veg Mini	2026-09-01	"01/09/2026" is ambiguous; read as day/month/year
3	Ravi Kumar New	9200000002	Non-Veg Thali	2026-09-03	—
11	Neha Joshi New	9200000006	Veg Full Thali	2026-09-25	—
12	Farhan New	9200000007	Veg Mini	2026-09-25	—
13	Priya New	9200000008	Non-Veg Thali	2026-09-25	"09/25/2026" read as month/day/year

Preview of samples/messy-customers.csv : 11 rows read, 1 empty line and the "Notes" column ignored; 5 imported, 1 deduped, 5 rejected. Nothing saved yet.

Annapurna Tiffins

DashboardCustomersPlansBillsNotificationsImport

22 Sept 2026Demo OwnerLog out

Columns are matched by name (name/customer, phone/mobile, address, plan, start_date/start). Dates like 2026-09-01, 01/09/2026, 1 Sep 2026 and Sep 1, 2026 all work; ambiguous dd/mm are read day-first. Plans are matched to your plans by name.

Plan when blank (optional)

Reject rows without a plan

Start date when blank (optional)

dd-mm-yyyy

Preview (no changes)Import

Import complete. 11 rows read, 1 empty line(s) ignored, ignored columns: Notes.

5

Imported

1

Deduped

5

Rejected

ROW	CUSTOMER	PHONE	PLAN	START	NOTES
2	Asha Iyer New	9200000001	Veg Mini	2026-09-01	"01/09/2026" is ambiguous read as day/month/year
3	Ravi Kumar New	9200000002	Non-Veg Thali	2026-09-03	—
11	Neha Joshi New	9200000006	Veg Full Thali	2026-09-25	—
12	Farhan New	9200000007	Veg Mini	2026-09-25	—
13	Priya New	9200000008	Non-Veg Thali	2026-09-25	"09/25/2026" read as month/day/year

Imported tab: row number, cleaned name (new/existing), phone, plan, start date, warnings.

Annapurna Tiffins

DashboardCustomersPlansBillsNotificationsImport

22 Sept 2026Demo OwnerLog out

Customer Name,Mobile No,address,Real Plan,start Date,Notes
ASHA IYER,+91 92000 00001,"12, MD Road, Indiranagar",veg mini,01/09/2026,caps name + +01 phone + dd/mm date
Ravi Kumar,92000000002,,Non-Veg Thali,2026-09-03,dishes in phone
asha Iyer,92000000001,,Veg Mini,5 Sep 2026,duplicate phone of row 2
+9200000001,+08 Layout,veg Mini,2026-09-01,blank name
Nehra,,veg Mini,2026-09-01,blank phone
Kiran,92000000004,,Veg Mini,,blank start date
Tara,92000000005,,Pizza Plan,2026-09-01,unknown plan
09,12345,,veg Mini,10/02/2026,bad phone and impossible date

Columns are matched by name (name/customer, phone/mobile, address, plan, start_date/start). Dates like 2026-09-01, 01/09/2026, 1 Sep 2026 and Sep 1, 2026 all work; ambiguous dd/mm are read day-first. Plans are matched to your plans by name.

Plan when blank (optional)

Reject rows without a plan

Start date when blank (optional)

dd-mm-yyyy

Preview (no changes)Import

Import complete. 11 rows read, 1 empty line(s) ignored, ignored columns: Notes.

5

Imported

1

Deduped

5

Rejected

ROW	NAME	PHONE	WHY SKIPPED
4	Asha Iyer	9200000001	duplicate phone: kept row 2

Deduped tab: row 4 repeats row 2's phone.

Tiffin Tracker - Project Guide · page 27 of 44

Annapurna Tiffins

Dashboard
Customers
Plans
Bills
Notifications
Import

22 Sept 2026
Demo Owner
Log out

Columns are matched by name (name/customer, phone/mobile, address, plan, start_date/start). Dates like 2026-09-01, 01/09/2026, 1 Sep 2026 and Sep 1, 2026 all work; ambiguous dd/mm are read day-first. Plans are matched to **your plans** by name.

Plan when blank (optional)

Reject rows without a plan

Start date when blank (optional)

dd-mm-yyyy

Preview (no changes)
Import

Import complete. 11 rows read, 1 empty line(s) ignored, ignored columns: Notes.

5

Imported

1

Deduped

5

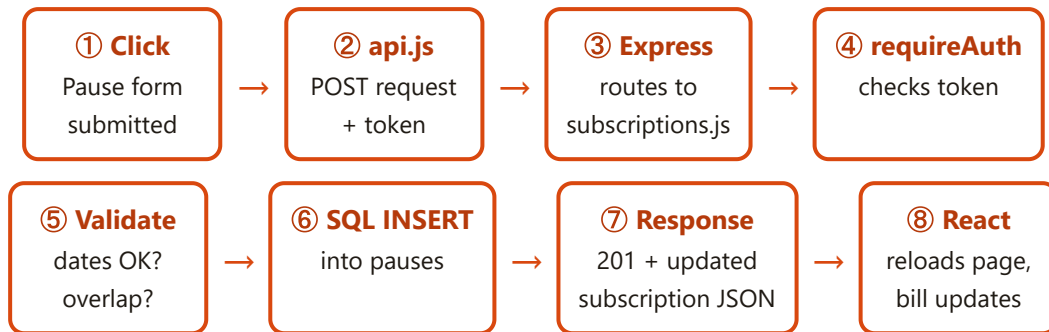
Rejected

ROW	DATA	PROBLEMS
5	∅ 9200000003 HSR Layout Veg Mini 2026-09-01	name is blank
6	Meena ∅ ∅ Veg Mini 2026-09-01	phone is blank
7	Kiran 9200000004 ∅ Veg Mini ∅	start date is blank
8	Tara 9200000005 ∅ Pizza Plan 2026-09-01	plan "Pizza Plan" does not match any of your plans
9	Om 12345 ∅ Veg Mini 31/02/2026	- phone "12345" is not a valid 10-13 digit number - start date "31/02/2026" is not a real date

Rejected tab: blank cells shown as ∅, and every problem listed per row (row 9 has two).

7 Backend: how a click becomes saved data

Let's follow one action, **pausing Aarav from 21 to 23 September**, through the whole system.



1. **Frontend:** the `PauseForm` component calls `api('/subscriptions/1/pause', {method: 'POST', body: {start_date: '2026-09-21', end_date: '2026-09-23', reason: ''}})`.
2. **api.js** adds `Authorization: Bearer <token>` and sends the request to the server.
3. **app.js** (Express) matches the URL to the subscription routes.
4. **requireAuth** (in `http.js`) verifies the token and learns the owner's id. No or invalid token → **401**.
5. **subscriptions.js** loads the subscription (only if it belongs to this owner, otherwise **404**) and checks:
 - dates are real dates in YYYY-MM-DD format → otherwise **400**
 - end is not before start, and the pause is within the subscription → otherwise **400**
 - no overlap with existing pauses → otherwise **409**
6. It runs `INSERT INTO pauses (subscription_id, start_date, end_date, reason) VALUES (...)`.
7. It replies **201 Created** with the subscription and all its pauses.
8. React reloads the customer and the bill preview (`GET /subscriptions/1/bill?month=2026-09`), which runs `computeBill` again, so the calendar and amount update.

Backend files explained

File	Responsibility
<code>index.js</code>	Opens the database, builds the app, starts listening on port 4000.
<code>app.js</code>	Registers middleware (JSON parsing, CORS, request logging), mounts each route group, serves the built website, and turns thrown errors into JSON error responses.
<code>db.js</code>	Creates the six tables if they don't exist; turns on foreign keys; <code>tx()</code> helper for transactions.
<code>billing.js</code>	<code>computeBill()</code> , the formula; <code>rangesOverlap()</code> , the overlap check. No database or clock inside, so it's easy to test.
<code>dates.js</code>	Is this a valid date? Is it a weekday? Add N days. First and last day of a month. All in UTC so time zones can't shift a day.
<code>http.js</code>	<code>HttpError</code> , <code>requireAuth</code> , <code>pageParams</code> (reads page/limit/sort/order safely), <code>paged</code> (builds the list response), phone clean-up.
<code>routes/auth.js</code>	Register, login, "who am I".
<code>routes/plans.js</code>	Create, list, edit, delete plans.

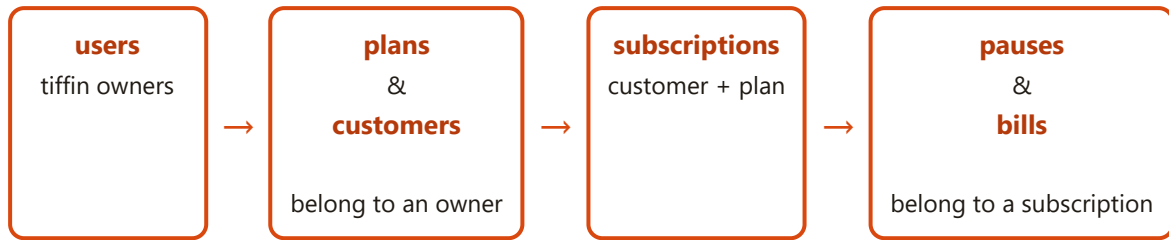
<code>routes/customers.js</code>	Customer CRUD, search/filter/sort/paginate, phone lookup, and the status SQL.
<code>routes/subscriptions.js</code>	Subscribe, pause, resume, remove pause, end subscription, bill preview.
<code>routes/bills.js</code>	Generate (save) bills for a month; list saved bills with summary.
<code>routes/dashboard.js</code>	Today's counts, tiffins to deliver, projected revenue, paused-today list.
<code>seed.js</code>	Deletes and recreates the demo owner with 3 plans and 32 customers.
<code>clock.js</code>	Simulated clock: load/save the date, move it, run each morning passed.
<code>notifications.js</code>	Notification Service (<code>send()</code> → outbox), the "due today" query, the morning job.
<code>importer.js</code>	Pure cleaning functions: CSV parser, header aliases, date and phone parsing, row validation.
<code>routes/simulation.js</code>	GET/POST /clock and GET /outbox.
<code>routes/import.js</code>	POST /api/import/customers: buckets rows, writes in one transaction.

Frontend files explained

File	Responsibility
<code>main.jsx</code>	Starts React and defines routes (URL → page), with Protected and GuestOnly guards.
<code>auth.jsx</code>	AuthProvider : remembers the logged-in user, offers login/register/logout to every page.
<code>api.js</code>	One function to call the backend (adds the token, handles errors); formats ₹ amounts and dates.
<code>useListQuery.js</code>	Stores search, filter, page, sort and order in the URL query string.
<code>components/ui.jsx</code>	Reusable pieces: StatusBadge, SortTh (sortable header), Pagination, Modal, ErrorNote, useDebounced.
<code>components/Layout.jsx</code>	Top bar and navigation around every app page.
<code>pages/*.jsx</code>	One file per screen (Chapter 5); <code>Notifications.jsx</code> , <code>ImportCustomers.jsx</code> and the Transfer dialog in <code>CustomerDetail.jsx</code> are covered in Chapter 6.
<code>styles.css</code>	All colours, spacing, tables, badges, modal, calendar and mobile rules.

8 The database, table by table

The database has **9 tables**: the 6 core ones below, plus **outbox**, **settings** and **clock_runs** for notifications. Think of each table as a spreadsheet; each row is one record. Tables are linked by **ids**.



users: the tiffin owners who log in

Column	Meaning
id	Unique number for the owner
name, business_name	Owner's name and tiffin service name (shown in the top bar)
email	Login email, unique (case doesn't matter)
password_hash	Scrambled password (bcrypt). The real password is never stored.
created_at	When the account was created

plans: what the owner sells

Column	Meaning
owner_id	Which owner this plan belongs to
name, description	"Veg Mini", "3 rotis, dal, sabzi"
price_paise	Monthly price in paise (250000 = ₹2,500)
is_active	1 = available for new subscriptions, 0 = retired

customers: people who receive tiffins

Column	Meaning
owner_id	Which owner's customer this is
name, phone, address	Contact details. (owner_id, phone) is unique : no duplicate phones per owner.
created_at	"Added" date shown on the Customers page

subscriptions: a customer on a plan for a period

Column	Meaning
customer_id, plan_id	Who, and which plan
price_paise	Price copied from the plan when subscribing
start_date	First delivery day
end_date	Last delivery day, or empty (NULL) if still running

transferred_from_id	If this subscription was taken over from someone, the id of their subscription (Level 2)
---------------------	--

pauses: days not to deliver

Column	Meaning
subscription_id	Which subscription is paused
start_date	First paused day
end_date	Last paused day (inclusive), or NULL = until resumed
reason	Optional note: Travel, Diwali, Unwell...

bills: saved month-end results

Column	Meaning
subscription_id, month	Whose bill and for which month ("2026-09"). One bill per subscription per month.
working_days	Weekdays in the month (e.g. 22)
subscribed_days	Weekdays inside the subscription period
paused_days	Subscribed weekdays that were paused
delivered_days	subscribed_days – paused_days
price_paise, amount_paise	Price used and final bill amount
generated_at	When it was calculated ("Last generated")

outbox: every message sent through the Notification Service (Level 1)

Column	Meaning
owner_id, customer_id, subscription_id	Who the message is about
channel, recipient	"sms" and the phone number
type	delivery_due, subscription_transferred_out, subscription_transferred_in
for_date	Which day the message is for
message, status	The text, and "sent"
dedupe_key	Unique key (e.g. delivery_due:12:2026-09-21) that stops duplicates

settings and clock_runs (Level 1)

Table	Meaning
settings	Key/value store; <code>clock_date</code> holds the simulated date (absent = real date)
clock_runs	One row per morning processed: date, how many were due, how many were notified

Why isn't "status" a column? Status (active/paused/upcoming/inactive) is **calculated** each time from today's date, the subscription dates and the pauses. Stored flags go out of date; calculated status never does.

How status is decided

Status	Rule (checked in this order, for today's date)
paused	Has a running subscription today and a pause covering today
active	Has a running subscription today, not paused
upcoming	Only has a subscription that starts in the future
inactive	No current or future subscription (never subscribed, or ended)

9 The REST API, endpoint by endpoint

All addresses start with `/api`. Except register, login and health, every request needs the header `Authorization: Bearer <token>`. Dates are `YYYY-MM-DD`, months are `YYYY-MM`, and money is in paise.

Auth

Method	Endpoint	What it does
POST	<code>/api/auth/register</code>	Create an owner account → returns token + user
POST	<code>/api/auth/login</code>	Log in → returns token + user
GET	<code>/api/auth/me</code>	Who is logged in

Plans

Method	Endpoint	What it does
GET	<code>/api/plans</code>	List plans (search, page, limit, sort = name/price/subscribers/created_at, order)
POST	<code>/api/plans</code>	Create a plan
GET / PUT / DELETE	<code>/api/plans/:id</code>	Read / update / delete one plan

Customers

Method	Endpoint	What it does
GET	<code>/api/customers</code>	List, search and filter customers (search, status, page, limit, sort = name/phone/plan/status/created_at, order, as_of)
GET	<code>/api/customers/phone/:phone</code>	Find a customer by phone
POST	<code>/api/customers</code>	Add a customer
GET	<code>/api/customers/:id</code>	Customer with all subscriptions and pauses
PUT / DELETE	<code>/api/customers/:id</code>	Edit / delete customer

Subscriptions, pause and resume

Method	Endpoint	What it does
POST	<code>/api/customers/:id/subscriptions</code>	Subscribe customer to a plan { plan_id, start_date }
GET	<code>/api/subscriptions/:id</code>	Subscription with pauses
POST	<code>/api/subscriptions/:id/pause</code>	Pause { start_date, end_date?, reason? }
POST	<code>/api/subscriptions/:id/resume</code>	Resume { date } (first delivery day again)
DELETE	<code>/api/pauses/:id</code>	Remove a pause
POST	<code>/api/subscriptions/:id/end</code>	End subscription { end_date }

GET	/api/subscriptions/:id/bill?month=	Live bill preview (not saved)
-----	------------------------------------	-------------------------------

Bills, dashboard and health

Method	Endpoint	What it does
POST	/api/bills/generate	Calculate and save bills for { month }
GET	/api/bills	List saved bills (month, search, page, limit, sort = name/phone/plan/amount/delivered/paused, order) + summary
GET	/api/dashboard	Today's counts, tiffins today, projected revenue, paused today
GET	/api/health	Is the server alive? → { ok: true }

Level 1: clock and outbox

Available at the root (/clock , /outbox) and under /api . Login is optional unless `PUBLIC_SIM_ENDPOINTS=false` ; with login, the outbox shows only your messages.

Method	Endpoint	What it does
GET	/clock	{ today, simulated, real_today }
POST	/clock	{ } run today again · { date } jump · { advance_days } move forward · { reset: true } real date. Runs the morning job and returns per-morning { date, delivery_day, due, notified, already_notified }
GET	/outbox	Sent messages (date, type, customer_id, search, page, limit, sort, order)

Level 2: transfer

Method	Endpoint	What it does
POST	/api/subscriptions/:id/transfer	{ effective_date, to_customer_id } or { effective_date, customer: { name, phone, address } } → { from, to, created_customer, billing_split }

Level 3: import

Method	Endpoint	What it does
POST	/api/import/customers	{ csv } or { rows } or raw text/csv; options default_plan_id, default_start_date, dry_run → { imported, deduped, rejected, total_rows, blank_rows_ignored, unknown_columns, details }

HTTP status codes you'll see

Code	Name	When it happens here
200	OK	Request worked (read, update, resume, generate)
201	Created	Something new was saved (register, customer, plan, subscription, pause)
204	No Content	Delete worked, nothing to return
400	Bad Request	Invalid input: bad date, missing name, end before start...

401	Unauthorized	Not logged in, wrong password, or expired token
404	Not Found	Doesn't exist, or belongs to another owner
409	Conflict	Clashes with existing data: duplicate phone/email, overlapping pause, second subscription, deleting a used plan
500	Server Error	Unexpected bug; details are printed in the server terminal

Example: a real request and response

```
POST /api/bills/generate
Authorization: Bearer eyJhbGciOi...
Content-Type: application/json

{ "month": "2026-09" }

→ 200 OK
{ "month": "2026-09", "bills_generated": 30, "total_amount_paise": 8393182 } ← ₹83,931.82
```

10 Security, validation and errors

Login and passwords

- Passwords are stored as a **bcrypt hash**, a one-way scramble. Even someone who reads the database can't see real passwords. At login, the typed password is hashed again and compared.
- After login the server issues a **JWT** (JSON Web Token), valid for 7 days, signed with a secret (`JWT_SECRET`). The browser saves it in **localStorage** and sends it with every request.
- If the token is missing, altered or expired, the server replies 401 and the website logs you out automatically.

Keeping owners' data apart (multi-tenancy)

Every query includes `owner_id = the logged-in owner`. Asking for another owner's customer returns 404, as if it doesn't exist, so ids of other businesses are never revealed.

Input validation

- Names are required; emails must look like emails; passwords need at least 8 characters.
- Phones: spaces, dashes and brackets are removed, then the number must be 10–13 digits (optional leading +).
- Dates must be real calendar dates (2026-02-30 is rejected).
- Prices must be positive whole paise.
- Page is at least 1; limit is between 1 and 100.

Safe sorting (SQL injection protection)

Sort options are chosen from a fixed list (**whitelist**). If someone sends `sort=name;DROP TABLE users`, it's ignored and the default sort is used. All other values go into SQL as **parameters**, never pasted into the SQL text.

Transactions

Multi-step changes (generating a month's bills, ending a subscription and trimming its pauses) run in a **transaction**: either every step is saved, or none are.

Consistent error format

```
{ "error": "Validation failed", "details": { "phone": "Phone must be 10-13 digits (optionally starting with +)" } }
```

The website shows `error` in a red box and lists each `details` message under it.

11 Glossary: every term explained

Business terms (what you see on the website)

Tiffin: a packed home-style lunch delivered in a lunch box.

Owner: the person running the tiffin service; the only user who logs in.

Tiffin service name / Business name: shown in the top-left of the app, e.g. "Annapurna Tiffins".

Plan / Meal plan: a product with a monthly price, e.g. "Veg Mini ₹2,500/month".

Available / Retired (plan): available plans can get new subscribers; retired ones can't but keep existing subscribers.

Subscribers: number of customers currently on a plan.

Customer: a person receiving tiffins; identified by phone number.

Subscription: a customer signed up to a plan from a start date until an optional end date.

Subscribe: create a subscription for a customer.

End subscription: set the last delivery day; the customer stops receiving and being billed after it.

Pause: a range of dates when no tiffin is delivered and nothing is charged.

Schedule pause: add a pause that starts in the future.

Pause until resumed / open-ended pause: a pause with no end date.

Resume: end the current pause; the chosen date is the first delivery day again.

Remove (pause): delete a pause entered by mistake; those days become billable.

Reason: optional note for a pause (Travel, Festival, Unwell).

Status: a customer's state today: Active, Paused, Upcoming or Inactive.

Active: subscribed and receiving tiffin today.

Paused: subscribed, but paused today.

Upcoming: subscription starts in the future.

Inactive: no current or future subscription.

Delivery day: a weekday (Mon–Fri). Weekends are not delivery days.

Tiffins to deliver today: count of Active customers on a delivery day.

Weekdays in month / Working days: Monday–Friday dates in the month (20–23).

Subscribed weekdays: weekdays within the customer's subscription period in that month.

Weekdays column (e.g. 22/22): subscribed weekdays / weekdays in the month.

Paused weekdays / Paused days: subscribed weekdays that fell inside a pause.

Delivered days: subscribed weekdays – paused weekdays; what the customer pays for.

Pro-rated bill: a bill reduced in proportion to the days actually served.

Bill / Amount: $\text{price} \times \text{delivered} \div \text{weekdays in month}$.

Bill preview: live calculation on the customer page; not saved.

Generate bills: calculate and save bills for all customers for a month.

Regenerate: recalculate saved bills after changes (safe to repeat).

Total to collect: sum of all bills for the month.

Projected bill total: dashboard estimate of this month's bills using current data.

Last generated: timestamp of the latest bill generation.

Delivery calendar: colour-coded month view: delivered, paused, weekend, not subscribed.

Paise: 1/100 of a rupee. Money is stored in paise to avoid rounding errors.

Simulated clock / simulated date: a "today" set by the owner (or a tester) instead of the real date; the whole app follows it.

Morning job: the step that finds customers due a delivery on a date and notifies them.

Due a delivery: subscription running that day, the day is a weekday, and not paused.

Notification: a message to a customer (delivery today, transferred in/out).

Notification Service: the single place all messages go through (`send()`).

Outbox: the list of every message sent.

Already sent: customers skipped because they were notified for that date before.

Transfer: handing a running subscription to another customer from a chosen date.

Effective date: the new holder's first delivery day; the old holder's last day is the day before.

Holder: the customer a subscription currently belongs to.

Mid-cycle: during a subscription, after it started and before it ended.

Carry over: the plan, locked price and end date continue unchanged for the new holder.

Billing split: the month's bill divided by who was served which days.

Taken over from / Transferred to: notes linking the two halves of a transferred subscription.

Import: creating customers and subscriptions from a spreadsheet (CSV).

CSV: "comma-separated values", a plain-text spreadsheet format every spreadsheet app can save.

Imported / Deduped / Rejected: the three outcomes of an import row (created / duplicate skipped / invalid).

Duplicate (dup) phone: the same phone number appearing more than once.

Preview / dry run: show the import report without saving anything.

Default plan / default start date: values used when those cells are blank.

Warning: a note on an imported row, e.g. an ambiguous date read day-first.

Ambiguous date: a date like 03/04/2026 that could be 3 April or 4 March.

Demo data / Seed: sample owner, plans and customers created by `npm run seed`.

Website (frontend) terms

Frontend / Client: the part that runs in the browser (the `client/` folder).

React: a JavaScript library for building interfaces from reusable **components**.

Component: a reusable piece of UI, e.g. `StatusBadge`, `Pagination`, `Modal`.

Page: a component shown for a URL, e.g. `Customers.jsx` for `/app/customers`.

JSX (.jsx): JavaScript with HTML-like tags, used by React.

State: data a component remembers while you use it (form values, loaded list).

Hook: a React helper function starting with "use", e.g. `useState`, `useEffect`, `useListQuery`.

Context / AuthProvider: shares the logged-in user with every page.

Route / React Router: maps a URL to a page without reloading the browser.

SPA (Single-Page Application): the site loads once; pages then switch instantly using JavaScript.

Protected route: a page that requires login; otherwise it redirects to `/login`.

GuestOnly route: login/register pages that redirect to the app if you're already logged in.

Layout: shared frame (top bar and navigation) around app pages.

Modal: a pop-up dialog over the page (Add customer, Pause, Resume...). Closes with X, Cancel, Esc or clicking outside.

Badge: small coloured label (status, plan availability).

Callout: highlighted message box with an action, e.g. "No bills generated yet. Generate now".

Alert / ErrorNote: red box showing an error message from the server.

Search: filter rows by typed text (name, phone, address).

Debounce: wait until the user stops typing (0.3 s) before searching.

Filter / Status tabs: show only customers with a chosen status.

Sort / Order (asc, desc): arrange rows by a column; ascending ▲ or descending ▼.

Pagination / Page / Limit: split long lists into pages; limit = rows per page.

Query string: the part of a URL after "?", e.g. `?status=paused&page=2`.

localStorage: small browser storage where the login token is kept.

Responsive design: layout adapts to phone, tablet and desktop widths.

Landing page: public marketing page explaining the product.

CTA (Call to action): a prominent button like "Start free".

Vite: tool that runs the React dev server (port 5173) and builds the production files.

Build (npm run build): turns source code into optimised files in `client/dist`.

Proxy: in dev mode, Vite forwards `/api` calls from port 5173 to the backend on 4000.

Backend and database terms

Backend / Server: the program that handles requests and talks to the database (`server/`).

Node.js: runs JavaScript outside the browser.

Express: a Node.js library for building web servers and APIs.

API: a set of addresses (endpoints) the frontend can call to get or change data.

REST: a style of API using URLs for things (`/customers/5`) and HTTP methods for actions.

Endpoint: one API address + method, e.g. `POST /api/customers`.

HTTP methods: GET = read, POST = create or perform an action, PUT = update, DELETE = remove.

Request / Response: the message sent to the server, and the server's reply.

JSON: text format for data, e.g. `{"name": "Asha", "phone": "9876543210"}`.

Status code: number in a response saying what happened (200, 201, 400, 401, 404, 409, 500).

Header: extra information sent with a request, like `Authorization`.

Route / Router: code handling a group of endpoints (e.g. `routes/customers.js`).

Middleware: code that runs before route handlers, e.g. JSON parsing, request logging, the `requireAuth` login check.

CORS: browser rule about which websites may call an API; enabled so the dev site can call the API.

Port: a numbered "door" on your computer: 4000 for the API, 5173 for the dev website.

localhost: your own computer.

Authentication: proving who you are (email + password → token).

Authorization: what you're allowed to access (only your own data).

JWT (JSON Web Token): a signed "ID card" the server gives after login; sent as `Bearer <token>`.

JWT_SECRET: secret key used to sign tokens. Set your own in production.

bcrypt / Hash: one-way scrambling of passwords so they can't be read back.

Validation: checking input is correct before saving.

Multi-tenant / owner scoping: many businesses share one app, each seeing only their data.

Database: organised permanent storage for data.

SQLite / node:sqlite: a lightweight database stored in a single file, built into Node.js 22.13+.

SQL: the language for querying databases (SELECT, INSERT, UPDATE, DELETE).

Table / Row / Column: a spreadsheet-like collection / one record / one field.

Schema: the design of all tables and columns.

Primary key (id): unique number identifying a row.

Foreign key: a column pointing to another table's id (e.g. `pauses.subscription_id`).

Unique constraint: prevents duplicates (email per user; phone per owner; one bill per subscription per month).

Index: a lookup structure that makes searches faster.

NULL: "no value", e.g. an end date that isn't set.

Cascade delete: deleting a customer also deletes their subscriptions, pauses and bills.

Transaction: a group of changes saved all-or-nothing.

Upsert: insert a row, or update it if it already exists (used when regenerating bills).

Idempotent: doing it twice gives the same result as once (Regenerate).

Snapshot: a saved copy of a result at a moment (saved bills).

Derived value: calculated on demand instead of stored (status).

Pure function: output depends only on inputs, no database or clock (`computeBill`).

UTC: universal time zone used for date maths so days never shift.

ISO date (YYYY-MM-DD): date format like 2026-09-17, which sorts correctly as text.

as_of: optional parameter to see statuses on another date (useful for testing).

Parameterized query: values sent separately from SQL text, preventing SQL injection.

SQL injection: an attack sneaking SQL commands into input; blocked by parameters and whitelists.

Whitelist: fixed list of allowed values (sort columns).

WAL (tiffin.db-wal, -shm): helper files SQLite uses for safe, fast writes. Normal to see.

Seed script: creates demo data (`server/src/seed.js`).

Environment variable: setting passed to the program (PORT, JWT_SECRET, DB_FILE).

npm / package.json: Node's package manager / file listing dependencies and commands.

Dependency: an external library the project uses (express, bcryptjs, jsonwebtoken, react...).

Idempotency key / dedupe_key: a unique value per message so repeating a job never sends twice.

Linked subscription (transferred_from_id): a column pointing from the new holder's subscription to the old one.

Normalisation / canonical phone: turning different ways of writing the same value into one standard form.

Header alias: an alternative column name that means the same field ("Mobile No" = phone).

ISO date: YYYY-MM-DD, the format all dates are converted to.

PUBLIC_SIM_ENDPOINTS: setting that allows /clock and /outbox without login (true by default, for testing).

Unit test: tests one small piece (billing formula) in isolation.

API / end-to-end test: starts the real server and runs a full user story through HTTP.

12 Running, testing and troubleshooting

First-time setup

```
npm run install:all    # install backend + frontend libraries
npm run seed          # create demo data (demo@tiffin.app / demo1234)
npm run build         # build the website
npm start             # start → open http://localhost:4000
```

Development mode (auto-reload while editing)

```
npm run dev:server    # terminal 1 → API on http://localhost:4000
npm run dev:client    # terminal 2 → website on http://localhost:5173
```

Tests

```
npm test              # 22 tests → expect "pass 22, fail 0"
```

billing.test.js checks the formula on tricky cases: full month, mid-month start, weekday pause, weekend-only pause, open-ended pause, pause across two months, fully paused month, ended subscription, future subscription, leap-year February, multiple pauses.

features.test.js checks Level 1 (who is notified, weekends, pauses, no duplicates), Level 2 (transfer, locked price, split bills, guards) and Level 3 (import counts, row numbers, dry run, re-import).

importer.test.js checks date, phone and CSV cleaning rules one by one.

api.test.js runs a whole owner story against a temporary in-memory database, including error cases and data separation between owners.

Command reference

Command	What it does
<code>npm run install:all</code>	Installs dependencies for server and client
<code>npm run seed</code>	Resets the demo owner's data
<code>npm run build</code>	Builds the React site into client/dist
<code>npm start</code>	Runs the server (API + built website) on port 4000
<code>npm run dev:server</code>	API with auto-restart on code changes
<code>npm run dev:client</code>	Vite dev website with instant reload
<code>npm test</code>	Runs all automated tests
<code>Ctrl + C</code>	Stops the running server in the terminal

Troubleshooting

Problem	Fix
No such built-in module: <code>node:sqlite</code>	Node.js is too old. Install Node 22.13 or newer (check with <code>node -v</code>).

<code>EADDRINUSE :4000</code>	Another server is already running on port 4000. Close that terminal, or start with a different PORT.
Website shows old design after code changes	Run <code>npm run build</code> again, then restart <code>npm start</code> .
Logged out unexpectedly / 401 errors	Token expired (7 days) or JWT_SECRET changed. Log in again.
Bills page empty	Bills are saved only after clicking Generate bills for that month.
Bill doesn't reflect a recent pause	Saved bills are snapshots. Click Regenerate . (The customer page preview is always live.)
Header shows 🕒 with a date / statuses look "wrong" for today	The app is on a simulated date. Notifications → Back to real date .
Import rejects every row	Check the header row has name and phone columns, and plan names match your Plans page.
No ⇌ Transfer button	The subscription was already transferred, or there is no running subscription.
Want a clean start	Stop the server, delete <code>server/data</code> , run <code>npm run seed</code> , start again.
Browser closes while typing in forms	A browser or extension crash, not the app. Try a private window, disable extensions, or use another browser.
See what the server is doing	Watch the terminal: each API call is logged as <code>METHOD url -> status (time)</code> .

Quick demo script (5 minutes): log in → Dashboard shows today's tiffins → look up 9845000000 → Aarav is paused, bill ₹2,045.45 → click Resume (today) → status Active, bill goes up → Pause 21–23 Sep → bill goes down → Bills → Generate → sort by Amount ▼ → search "Nair" → Notifications → Jump to next Monday → Aarav → ⇌ Transfer to a new customer → Import → Load messy sample → Preview → Import.